Installation

Deadline : April 11/18/19, 2024 (your exercise session, cf. details below)

How to complete a project successfully?

Follow the rules as described in the Lecture!

GitLab at TUHH[1] is used as the submission system in this course. Your GitLab credentials are the same as for StudIP. If you cannot log into GitLab with your StudIP login and password, contact the teaching assistant immediately. All your code must be pushed to your group repository provided to you by the teaching assistant. Remember, that each group member needs to demonstrate their contributions through code commits. You can update your solution until the deadline. The latest commit before the deadline is considered by default. If you want your late submission to be graded for penalized maximal number of possible points, the **whole** group must send an email to the teaching assistant **before** the interview day.

How to get additional information?

You are encouraged to discuss past and present project sheets with the teaching assistants. Either approach the teaching assistant during the exercise session, or visit us during the weekly office hours. We are also available through e-mail or on the StudIP forum. We try to reply as quickly as possible and in general, you should get a reply the next weekday, but we cannot guarantee this.

[1] https://collaborating.tuhh.de/

Installation

In this course, you will develop a compiler for a subset of C++. You will implement your compiler either in Haskell[2] or in C++[3].

We will use the following software:

- GHC[4]
- cabal[5]
- stack[6]
- BNFC[7]
- llvm[8]
- make[9]
- alex (optional, for Haskell)[10]
- happy (optional, for Haskell)[11]
- llvm-hs (optional, for Haskell)[12]
- clang (optional, for C++)[13]
- bison (optional, for C++)[14]
- flex (optional, for C++)[15]

The `bnfc` tool is used to generate a parser and code snippets for the implementation language of your choice, and the `make` tool to control the compilation process. The `bnfc` tool is written in the Haskell programming language and, therefore, you will need a working Haskell installation to compile `bnfc`. We recommend that you use Visual Studio Code[16] (`code`) to develop inside a

[2] http://www.haskell.org
[3] http://isocpp.org/
[4] http://www.haskell.org/ghc
[5] http://www.haskell.org/cabal
[6] http://www.haskellstack.or
[7] http://bnfc.digitalgrammars.com/
[8] http://www.llvm.org
[9] https://www.gnu.org/software/make/
[10] http://www.haskell.org/alex/
[11] http://www.haskell.org/happy/
[12] https://github.com/llvm-hs/llvm-hs
[13] http://clang.llvm.org/
[14] https://www.gnu.org/software/bison/
[15] http://flex.sourceforge.net/
[16] https://code.visualstudio.com

container[17]. The Docker[18] container image is provided by us and contains all aforementioned tools.

Please follow the official installation instructions[19] to install Docker, `code`, and the Dev Containers extension. If you're using macOS, you may have to adjust some security settings for Docker. In the system preferences, go to *Security & Privacy* → *Privacy* → *Full Disk Access* and grant the respective permissions to Docker.

Then, open the directory with the contents from `0-installation.zip` (the directory this PDF file probably is located in) with `code`. Because of the included `.devcontainer` directory, `code` should prompt you with the option to open the directory in a dev container, which you should confirm. This will take a while because the container needs to be built for the first time. Finally, if you successfully opened the directory in the devcontainer, you're ready to proceed with the tasks.

In case of problems, please consult the documentation (cf. links in footnotes). If that doesn't help, you can use the StudIP forums or ask the teaching assistant for help. In any case, please provide a description of your problem, including error messages, screenshots, ...

Tasks

Task 1 Test your BNFC installation using `bnfc -m CPP.cf`. This command will generate Haskell code as well as a makefile that you can use to compile with the command `make`. Alternatively, you may also generate code for the language of your choice (e.g., C++). Demonstrate the application of the `TestCPP` executable to the (enclosed) "foo.cc" sample file (first run `make`, this should compile the `TestCPP` application, which you should then execute with `./TestCPP foo.cc`).
Note that to pass, you have to demonstrate your working toolchain to the teaching assistant in class.
Deadline: your group's exercise session on April 11 or 18.

Task 2 Depending on your language of choice, consider the corresponding (enclosed) llvm demo (either "llvm_cpp" for C++, or "llvm_hs" for Haskell). Demonstrate that you can compile and run the demo using `make test`.
Note that to pass, you have to demonstrate your working toolchain to the teaching assistant in class.
Deadline: your group's exercise session on April 11 or 18.

Task 3 As soon as the project groups are fixed, and the teaching assistant sent you information on your group's `git` repository on GitLab[20], create a *commit* in that repository.
Deadline: April 19, 8 AM.

[17] https://code.visualstudio.com/docs/devcontainers/containers
[18] https://docker.com
[19] https://code.visualstudio.com/docs/devcontainers/containers#_installation
[20] https://collaborating.tuhh.de

Grading

You can earn up to 5 points in this project phase. Up to 5 points are awarded for completing all 3 tasks. You *must* complete task 3 to get any points. Note, that every group member should be able to present a working development environment. Points are awarded individually.